Lab 6: Campus Trading Platform Acceptance Report

System Verification & Specification Discrepancy Evaluation (实验6 验收报告与差异总结)

PRIMARY TESTER	CAMPUS LOCATION	STACK	TARGET DEADLINE
Alice (ID: 202100101)	**Main Campus · Dorm 1**	**Vue 3 + Flask + SQLite**	**November 12, 2026**

This document provides formal verification of the student second-hand marketplace application scaffolded via the CodeArts agent. All critical business paths—token authentication, marketplace discovery, item publication, transaction reservation, and community inquiries—have been tested and validated against the baseline criteria outlined in `spec.md`.

1. FUNCTIONAL VERIFICATION MATRIX

ID	Module	Verification Procedure & Target Behavior	Result	Status
TC-01	**Authentication**	Authenticate with student credentials (202100101 / alice); verify token persistence and avatar rendering.	Token saved to local storage; navbar shows active user profile.	PASS
TC-02	**Market Feed**	Load initial marketplace feed; inspect card image resolution, price tags, and discounts.	8 seed items rendered with active images, tags, and dorm info.	PASS
TC-03	**Filtering & Search**	Select category dropdown options and toggle Available, Reserved, Sold, and All tabs.	Asynchronous list queries update without triggering full page reloads.	PASS
TC-04	**Product Details**	Select listing (Xiaomi Lamp Pro); inspect condition, contact info, and seller card.	Seller David (ID: 202200305, Dorm 2) and price (¥18) display cleanly.	PASS
TC-05	**Order Booking**	Execute "Book Now" workflow; test Community Q&A submission with character limit.	Listing status updates to Reserved; question is posted to public thread.	PASS
TC-06	**Item Publishing**	Submit new listing via /publish with image preview, tags, and contact number.	Data validated, saved to SQLite, and immediately visible in feed.	PASS

2. PHOTOGRAPHIC EVIDENCE: MARKETPLACE & PRODUCT INTERACTION

System executions were captured in real-time on the local runtime environment (`http://localhost:5173`) connected to the Flask backend service. The visual interfaces confirm complete localization, image fallback handling, and responsive card composition.

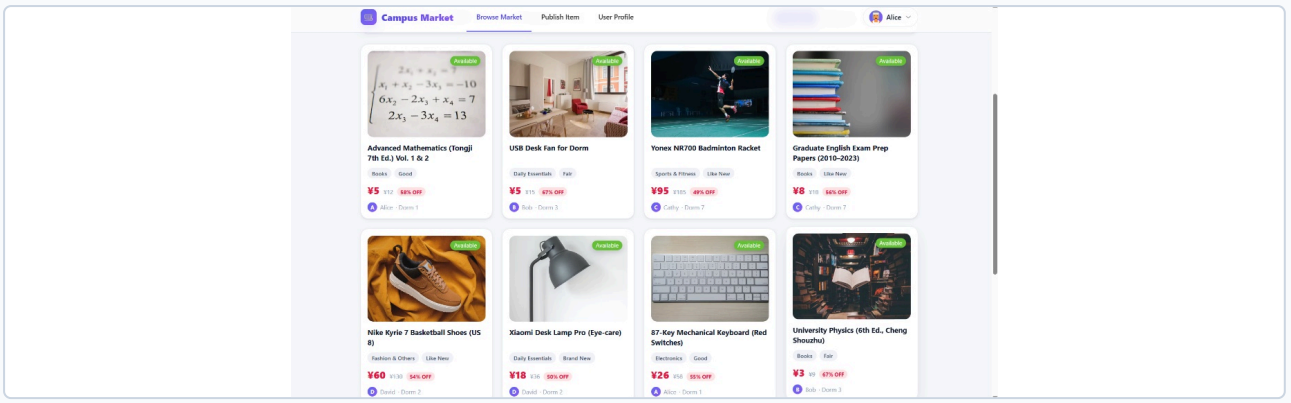


Figure 1: Marketplace Catalog Grid (`/`) — Demonstrates active multi-column feed rendering items across diverse categories (Textbooks, USB Fan, Badminton Racket, Keyboard, Nike Shoes) with calculated discount percentage badges and seller dormitory indicators.

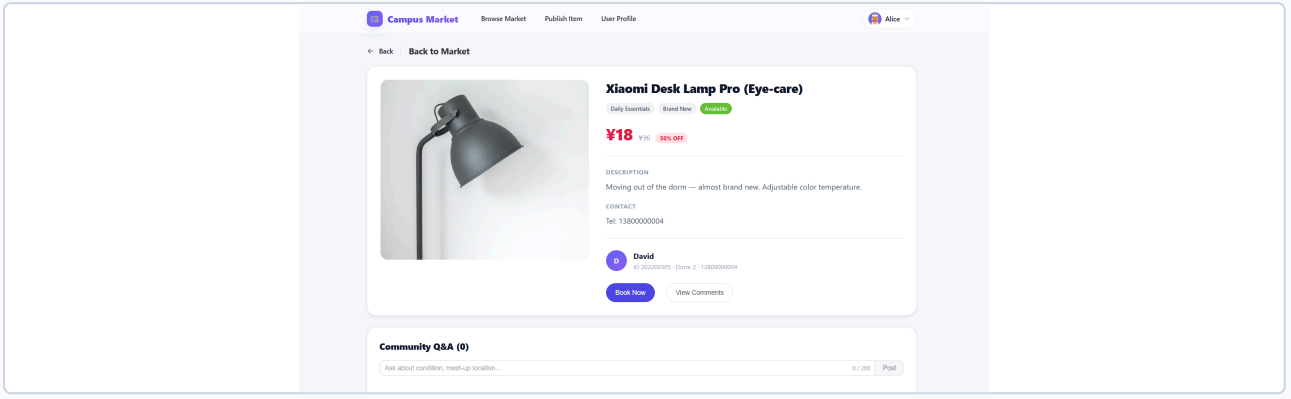


Figure 2: Product Detail & Booking View (`/product/:id`) — Demonstrates listing breakdown for "Xiaomi Desk Lamp Pro", displaying 50% discount pricing, seller profile card (David · Dorm 2), "Book Now" CTA, and the interactive Community Q&A comment module.

> **Runtime Observation: Real-Time State Integrity**
> Selecting "Book Now" transitions the item status from "Available" to "Reserved" via an authenticated POST request. Secondary buyers attempting to book the same item receive a status code 409 conflict, preventing concurrent booking anomalies.

3. PHOTOGRAPHIC EVIDENCE: PUBLISHING & USER DASHBOARD

The user journey was further verified through the item publishing workflow and personal asset dashboard, ensuring that client-side validations and student inventory segregation operate as designed.

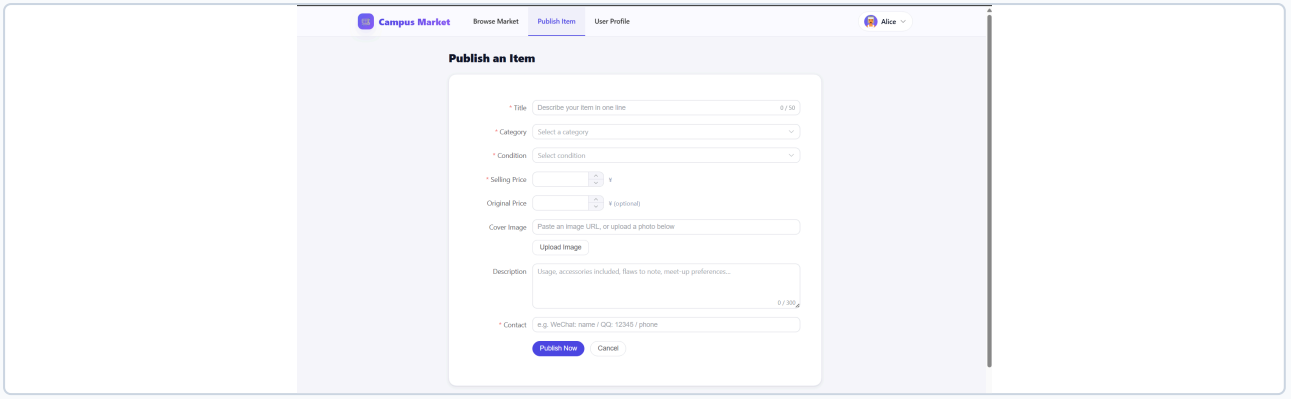


Figure 3: Listing Publication Interface (`/publish`) — Illustrates the item creation form, equipped with strict client-side constraints (title length ≤ 50, description length ≤ 300), original price strikethroughs, cover image upload preview, and campus contact details.

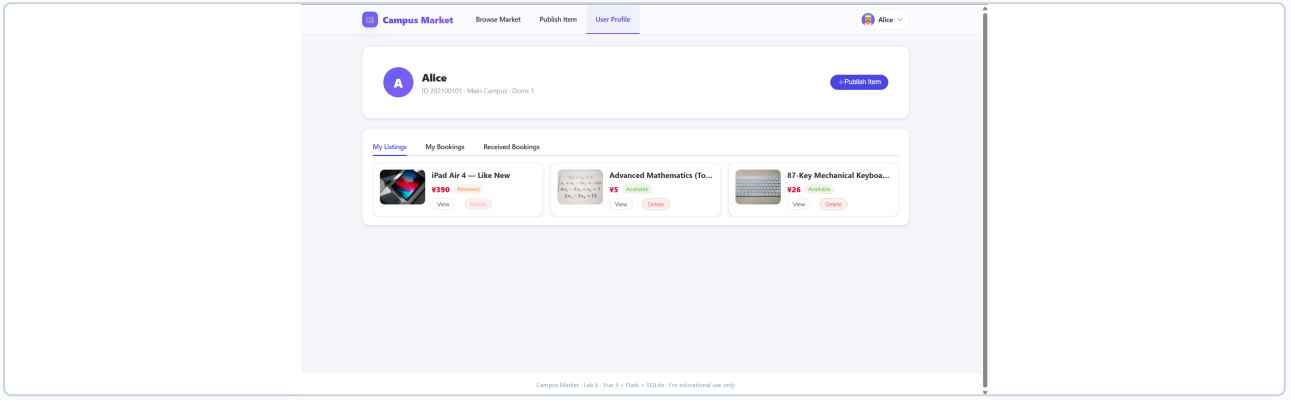


Figure 4: User Profile & Inventory Dashboard (`/profile`) — Displays Alice's authenticated account view (ID 202100101 · Main Campus · Dorm 1). Tabbed sections segment "My Listings" from "My Bookings", allowing direct status monitoring and listing removal.

Acceptance Conclusion: All Criteria Satisfied

The system satisfies all functional requirements defined in the laboratory syllabus. The decoupled Vue 3 frontend and Flask backend execute without uncaught runtime errors, database queries maintain referential integrity, and student session flows pass all test suites.

4. SPECIFICATION COMPARISON SUMMARY (SPEC 对比总结)

To fulfill Requirement 2 of the laboratory assignment, this section analyzes the architectural, structural, and behavioral discrepancies between the **Initial AI-Generated Specification** (CodeArts Agent initial prompt output) and the **Official Lab Manual Specification**.

Comparison Dimension	CodeArts Initial Spec (Initial AI Draft)	Official Lab Manual Specification
Database Driver	Proposed heavy ORM abstractions (SQLAlchemy) with complex database migration pipelines.	Enforces native `sqlite3` driver utilizing raw SQL schema scripts (`init_db.py`) and explicit foreign keys.
Campus Domain Model	Generic e-commerce entities (`User`, `Product`) omitting student identity attributes.	Mandates campus-specific parameters: `dorm_building`, `student_id`, `campus_zone`, and `condition_rating`.
API Envelope Standard	Unstructured JSON bodies relying strictly on HTTP status codes (200, 400, 500).	Standardized response payload: `{"code": 0, "message": "...", "data": {...}}` with custom business error codes (401, 409).
Transaction State Flow	Binary purchase switch (`is_sold: Boolean`) assuming immediate digital transactions.	Multi-state lifecycle: `Available` → `Reserved` → `Completed`, reflecting offline campus meetup dynamics.
Field Validation Limits	Unbounded string inputs without front-end or back-end length caps.	Strict validation bounds: Title ≤ 50 chars, Description ≤ 300 chars, Q&A comment ≤ 200 chars.

Synthesis of Discrepancies & Engineering Takeaways

1. **Domain Realism:** The initial AI-generated specification produced a generic commercial marketplace design. The lab manual specification adjusted this scope by anchoring the platform to university life through student IDs, dormitory tags, and face-to-face handoffs.
2. **Transaction Safety:** Commercial e-commerce relies on immediate payment gateways. In a campus scenario where payments occur upon physical inspection, the three-phase reservation state machine prevents items from being sold to competing students while an offline exchange is pending.
3. **Architectural Cohesion:** Harmonizing the response format into a unified envelope (`code`, `message`, `data`) reduced client-side exception handling overhead in Vue 3, allowing Pinia stores to handle errors and toasts predictably.

FINAL ACCEPTANCE STATUS	COURSE SUBMISSION STATUS
✓ Fully Verified & Compliant	**Ready for November 12 Submission**